

0.2 Project Rules

Real-Time Big Data Processing

Anton Dignös - anton.dignoes@unibz.it

Free University of Bozen-Bolzano

Organization

Groups:

- **one student** → default & typical situation, or
- **two students** → project proposal must justify need for two people, same per-student effort

Deliverables:

- **project proposal** → not evaluated, serves to check feasibility & to give you feedback
- **source code** → evaluated, runnable
- **written report** → evaluated, 10-15 pages (~ 3000 words)
- **slides** → evaluated, 15 min. (or 30 min. in case of 2 students) talk including demo, clear & to the point

Deadlines (T = chosen exam date):

- **T - 25 days** → 1 page project proposal (hard deadline, better to submit proposal earlier)
- **T - 7 days** → report & code submission (hard deadline, may optionally include slides)
- **T** → slides submission (if not done) & project presentation: 15" talk incl. demo + 5-15" questions

Source Code

Required components

- **Kafka** – must use at least a broker, other Kafka components (e.g., Connect) are optional
- **stream processor** – Flink (any API) or Spark (structured streaming)
- **web frontend** – must show some *live data*, any technology is fine
- *any programming language / data source / additional component* are OK
 - e.g., Docker (recommended), Kafka producers, backend apps ...
 - a streaming data source is generally required but may authorize *re-playing* of stored data
 - may reuse sources / architectures / technical solutions of labs, as long as project is original

Submitted as **git repository**

- <https://gitlab.inf.unibz.it/> repository (public or private, my username: dignoes)
- must include a README indicating how to run the code (even if not possible, e.g., private data)
- may indicate the tag/commit to evaluate
 - if omitted: last default branch commit made before the submission deadline

Project Proposal

1 (better) - 2 pages, 300 ± 100 words approx.

Required content (structure is free)

- **title & authors**
 - if two students, add some sentences at the end justifying why they are needed
- **application domain**
 - few sentences describing the considered scenario and what the project will do
- **data sources**
 - just list them if seen in the course, otherwise provide link/one sentence description
 - how the sources will be accessed? (in presence of multiple APIs / data access methods)
- **technologies & architecture**
 - list components, indicating the ones to be developed (with prog. language & libraries used)
 - include an architecture diagram (1 picture worth 1000 words)
- **functionalities**
 - esp. the ones accessed by end users on the frontend side

Project Proposal (cont'd)

Rationale

- force you to think about data sources & other relevant aspects impacting project feasibility
- give you some **feedback** and the “green light” prior to implementing the project
→ major variations after the proposal (e.g., change of data source) should be rediscussed

Submission

- via **Teams ‘assignment’** created prior to exams
- will answer and provide feedback to each proposal, as soon as I can after it is submitted
→ don't wait the last moment, the sooner the better!

Written Report

10–15 pages, 2000–3000 words approx. (max 4000 words) with following *required* structure:

0. Title page

→ title, authors, abstract, *link to the code repository*

1. Application domain

2. Description of the data sources

→ complexity, size, speed, ...

3. Technologies and overall architecture

→ technologies (languages, libraries/frameworks, ..., *few sentences each*)

→ system architecture with *required diagram*

→ workflow (i.e., data processing steps + any manual actions involved in using the system)

4. Functionalities

→ including screenshots for illustration

5. Lessons learned

→ e.g., what have worked? what not? what would you improve?

Written Report and Code Submission

Done via **Teams 'assignment'** and **git repository**

- assignment dedicated only to students whose proposal is approved (no proposal $\Rightarrow$ no exam)
- link the git repository in the report, and upload the report on Teams assignment
- will confirm successful reception of material (or ask to fix issues)

Evaluation

- register to the exam on date **T** and submit on Teams by **T - 7 days hard deadline**
- no submission, no exam
→ let me know if you aim for next exam keeping same proposal

Slides, Demo & Presentation

Format

- **15 min.** (one student) or **30 min.** (2 students) talk including demo (strict time limit)
- **5-15 min.** (one student) or **5-30 min.** (2 students) discussion

Slides

- submitted either with report or last moment prior to the talk via separate Teams 'assignment'
- should be concise and follow a structure similar to the one of the report

Demo

- suggest having the system *running and ready* prior to the talk (try it before!)

Questions

- entirely related to the project & also aiming at testing authorship (plagiarism ⇒ exam failed)
→ ok using third-party material, but acknowledge sources & don't present as own contribution!

Organization

- oral exam is public: anyone can attend in person (no streaming, no recording)
- exam agenda posted online with each group assigned a 30 min. (1 student) or 60 min. (2 students) time slot
- grades communicated after the presentation, usually the same day

Evaluation Criteria

20% *scenario, correctness of the solution and usability*

- scenario: is it “real-time (big) data processing”? is it a good fit for the course’s technologies?
- solution correctness: are required components present? is solution appropriate to scenario?
- solution usability: for both frontend user & system maintainer (e.g., how easy to run the system)

20% *quality of the software design and source code*

- proper code, README, config. & dependencies handling (think someone else will use the code)

20% *technical coverage, complexity of the project*

- complexity: how trivial/complex are the tasks tackled by the project?
- coverage: does the project (report / code) account for efficiency, scalability, fault tolerance, result correctness w.r.t. out-of-order/late data & faults?

20% *written report*

- completeness (w.r.t. discussing aforementioned aspects), structure, clarity, quality of writing

20% *presentation, including slides, demo, answers to questions*

- are slides, talk & demo clear and effective? did the system run? are answers satisfactory?

Possible Data Sources

Course labs:

- Wikipedia edit events [↗](#)
- RSS feeds via Kafka Connect [↗](#)
- OpenSky Network aircraft data [↗](#)
- Finnhub cryptocurrency trades [↗](#)
- Open Data Hub (ODH) API [↗](#)

Old course editions suggestions:

- UniBit realtime stock price [↗](#)
- USGS real-time data [↗](#)
- Aviation Edge air traffic data [↗](#)
- NYC's Citibike data [↗](#)
- Hacker News API [↗](#)

Past projects (*decreasing popularity*):

- social media: X (100 \$/m) [↗](#), Reddit (PRAW) [↗](#)
- stock data: AlphaVantage [↗](#), Finnhub [↗](#), Yahoo Fin. [↗](#)
- online games: RIOT Games API [↗](#)
- transportation: Entur API [↗](#), OpenSky Network [↗](#)
- RSS: Google News [↗](#), and others
- local data: OpenDataHub [↗](#)
- replay of static data: Kaggle and local datasets
- automatically-generated (e.g., sensor readings)
- weather: OpenWeather [↗](#)
- proprietary data

List of free APIs [↗](#) (*beware of rate limits / API keys*)